

ING CCV - 2^{ÈME} ANNÉE -

Module	Déploiement IaaS	Date	09 mai 2026
Enseignant	Mohamed Chiheb BEN CHAABANE	Durée	03 heures

DevOps CI/CD Lab: Terraform + Ansible + Docker + GitHub Actions

Prerequisites

- AWS Account
- GitHub Account
- SSH Key Pair
- Terraform installed locally
- Ansible knowledge (basic)

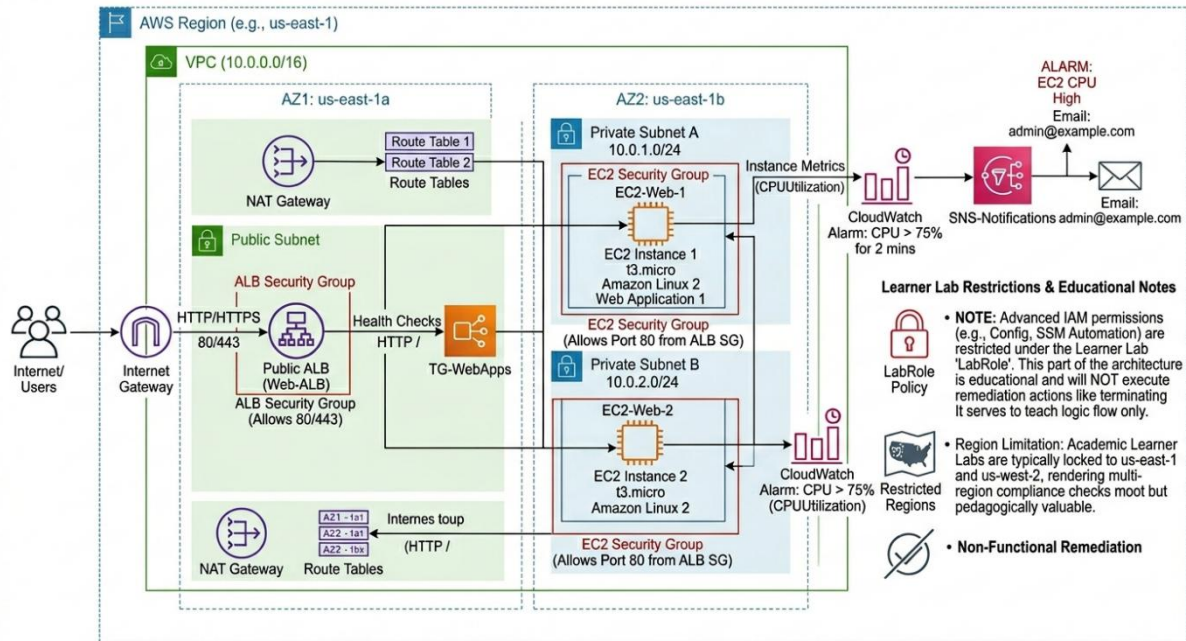
Objective: Learn to deploy cloud infrastructure and applications automatically using CI/CD.

Lab Objectives

By the end of this lab, students will:

- Provision AWS infrastructure using Terraform
- Configure servers using Ansible
- Deploy a containerized e-commerce app
- Automate everything via GitHub Actions

LAB AWS ARCHITECTURE



CI/CD Pipeline Workflow

GitHub Actions Pipeline



Terraform → AWS Infrastructure



EC2 Instances (private subnet)



Ansible → Docker + App Deployment



ALB → Nginx → Node.js → MongoDB

STEP 2 — Create GitHub Repository

Create a repo: *ecommerce-devops-lab*

STEP 3 — Project Structure

repo/

├── terraform/

├── ansible/

└── .github/workflows/pipeline.yml

STEP 4 — Terraform (Infra. Deployment)

Inside terraform/:

Key components:

- VPC
- Subnets
- ALB
- EC2 instances
- Security groups

Add Output (IMPORTANT)

YAML

```
output "instance_public_ips" {  
  value = aws_instance.web[*].public_ip  
}
```

👉 This is used later by Ansible dynamically.

STEP 5 — Ansible Resource Provisionning

Inside ansible/:

Playbook installs:

- Docker
- Docker Compose
- Runs containers:

- Nginx
- Node.js app (PM2)
- MongoDB

Run manually (for testing):

Shell

```
ansible-playbook -i inventory.ini deploy.yml
```

inventory.ini (sample file)

```
[linux_servers]
```

```
193.55.4.1 ansible_user=your_remote_username
```

```
193.55.4.2 ansible_user=your_remote_username
```

STEP 6 — Configure GitHub Secrets

Go to: **GitHub** → **Settings** → **Secrets** → **Actions**

Add:

Secret Name	Description
AWS_ACCESS_KEY_ID	AWS access key
AWS_SECRET_ACCESS_KEY	AWS secret
AWS_REGION	e.g. us-east-1
EC2_KEY	SSH private key

STEP 7 — Create GitHub Actions Pipeline

Create:

```
.github/workflows/pipeline.yml
```

FINAL PIPELINE (FULL VERSION)

YAML

```
name: Full DevOps Pipeline
```

on:

push:

branches: ["main"]

jobs:

=====

1. TERRAFORM INFRASTRUCTURE

=====

terraform:

name: Provision Infrastructure

runs-on: ubuntu-latest

defaults:

run:

working-directory: terraform

outputs:

instance_ips: \${{ steps.export.outputs.instance_ips }}

steps:

- name: Checkout Code

uses: actions/checkout@v3

- name: Configure AWS Credentials

uses: aws-actions/configure-aws-credentials@v2

with:

aws-access-key-id: \${{ secrets.AWS_ACCESS_KEY_ID }}

aws-secret-access-key: \${{ secrets.AWS_SECRET_ACCESS_KEY }}

region: \${{ secrets.AWS_REGION }}

- name: Setup Terraform

uses: hashicorp/setup-terraform@v3

- name: Terraform Init

run: terraform init

- name: Terraform Validate

run: terraform validate

- name: Terraform Plan

run: terraform plan -out=tfplan

- name: Terraform Apply

run: terraform apply -auto-approve tfplan

- name: Export Instance IPs

id: export

run: |

IPS=\$(terraform output -json instance_public_ips | jq -r '.[0]' | tr '\n' ',')

echo "instance_ips=\$IPS" >> \$GITHUB_OUTPUT

=====

2. ANSIBLE CONFIGURATION

=====

ansible:

name: Configure Servers

runs-on: ubuntu-latest

needs: terraform

steps:

- name: Checkout

uses: actions/checkout@v3

- name: Install Ansible

run: |

sudo apt update

sudo apt install -y ansible jq

- name: Setup SSH Key

run: |

echo "\${{ secrets.EC2_KEY }}" > key.pem

chmod 600 key.pem

- name: Generate Inventory

run: |

echo "[web]" > inventory.ini

for ip in \$(echo "\${{ needs.terraform.outputs.instance_ips }}" | tr ' ' ','); do

echo "\$ip ansible_user=ec2-user ansible_ssh_private_key_file=key.pem" >> inventory.ini

done

- name: Run Ansible Deployment

run: |

ansible-playbook -i inventory.ini ansible/deploy.yml

env:

ANSIBLE_HOST_KEY_CHECKING: "False"

=====

3. *OPTIONAL: TERRAFORM DESTROY*

=====

destroy:

name: Destroy Infrastructure (Manual)

if: github.event_name == 'workflow_dispatch'

runs-on: ubuntu-latest

defaults:

run:

working-directory: terraform

steps:

- uses: actions/checkout@v3

- uses: hashicorp/setup-terraform@v3

- name: Terraform Destroy

run: terraform destroy -auto-approve

STEP 8 — Run the Pipeline

Commit & push:

Shell

git add .

git commit -m "Deploy full pipeline"

git push origin main

STEP 9 — Observe Execution

Go to:

 **GitHub → Actions**

You will see:

1. Terraform creates infrastructure
2. EC2 instances are launched
3. Ansible connects via SSH
4. Docker containers start
5. App becomes accessible via ALB

EXPECTED RESULT

After pipeline finishes:

 Open ALB DNS → You see:

E-Commerce Store

- Laptop \$1200

- Phone \$800

LEARNING OUTCOMES

Students understand:

-  Infrastructure as Code (Terraform)
-  Config Management (Ansible)
-  Containers (Docker + PM2)
-  Reverse Proxy (Nginx)
-  CI/CD automation (GitHub Actions)
-  Cloud Architecture (AWS ALB + EC2)

⚠ COMMON ISSUES (Debug Guide)

Issue	Fix
SSH fails	Check security group (port 22)
App not loading	Check Nginx container
Terraform fails	Validate credentials
Ansible timeout	Ensure NAT / internet access

/. END ./